Introdução a Redes Neurais com TensorFlow e Keras

Importação das bibliotecas, classes e métodos utilizados ao longo da aula:

In [1]:
```
import os
import random
import pandas as pd
import numpy as np
from sklearn.preprocessing import OneHotEncoder
import tensorflow as tf
from tensorflow import keras
from sklearn.model_selection import train_test_split
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    classification_report,
    ConfusionMatrixDisplay,
    roc_auc_score,
    RocCurveDisplay)
import matplotlib.pyplot as plt
```

Definição de uma semente aleatória fixa:

In [2]:
```
RANDOM_STATE = 7895
```

In [3]:
```
os.environ["TF_DETERMINISTIC_OPS"] = "1"
random.seed(RANDOM_STATE)
np.random.seed(RANDOM_STATE)
tf.keras.utils.set_random_seed(RANDOM_STATE)
tf.config.experimental.enable_op_determinism()
```

Carregando os dados

Leitura do dataset a partir de um arquivo CSV armazenado em um repositório GitHub:

In [4]:
```
df = pd.read_csv('https://raw.githubusercontent.com/vqrca/ml-datasets/refs/heads/main/BankChurners_portugues.csv')
```

Exibição das primeiras observações para verificar a estrutura dos dados e as variáveis disponíveis:

In [5]:
```
df.head()
```

Out[5]:

	Numero_Cliente	Target	Idade_Cliente	Genero	Numero_Dependentes	Escolaridade	Estado_Civil	Faixa_Renda_Anual	Categoria_Carta
0	768805383	Ativo	45	Masculino	3	Ensino medio	Casado(a)	US60$mil a US$ 80 mil	Azı
1	818770008	Ativo	49	Feminino	5	Graduacao	Solteiro(a)	Menos de US$ 40 mil	Azı
2	713982108	Ativo	51	Masculino	3	Graduacao	Casado(a)	US80$mil a US$ 120 mil	Azı
3	769911858	Ativo	40	Feminino	4	Ensino medio	Nao informado	Menos de US$ 40 mil	Azı
4	709106358	Ativo	40	Masculino	3	Sem escolaridade formal	Casado(a)	US60$mil a US$ 80 mil	Azı

5 rows × 21 columns

Preparando os dados para a rede neural

Redes neurais **não interpretam valores categóricos**. Se codificássemos `Escolaridade` como 0, 1, 2, 3, a rede assumiria que existe uma **ordem e uma distância matemática** entre as categorias — o que raramente é verdade. Por isso, usamos o **`OneHotEncoder`** : cada categoria vira uma coluna binária independente (0 ou 1), sem impor qualquer relação artificial entre elas.

O alvo `Target` também é convertido para o formato numérico esperado pelo Keras: **0 para `Ativo` e 1 para `Inativo`** .

In [6]:
```
df_modelo = df.drop(columns=['Numero_Cliente'])

colunas_categoricas = [
    'Genero',
    'Escolaridade',
    'Estado_Civil',
    'Faixa_Renda_Anual',
    'Categoria_Cartao'
]

# Transforma variáveis categóricas em variáveis binárias
encoder = OneHotEncoder(sparse_output=False, handle_unknown='ignore')
encoded_data = encoder.fit_transform(df[colunas_categoricas])
encoded_df = pd.DataFrame(encoded_data, columns=encoder.get_feature_names_out(colunas_categoricas))

# Mantém o dataframe original mas remove as colunas categóricas que foram codificadas
df_encoded = pd.concat([df.drop(columns=colunas_categoricas), encoded_df], axis=1)

X = df_encoded.drop('Target', axis=1)
y = (df_encoded['Target'] == 'Inativo').astype(int)

target_names = ['Ativo', 'Inativo']
```

Verificamos as dimensões finais dos dados após a codificação. O número de colunas de `X` agora é maior do que o original, pois cada variável categórica foi expandida em várias colunas binárias:

In [7]:
```
print(f'Formato de X: {X.shape}')
print(f'Formato de y: {y.shape}')
```

```
Formato de X: (10127, 38)
Formato de y: (10127,)
```

Separando treino e teste

Assim como nos modelos anteriores, dividimos os dados em treino (80%) e teste (20%), mantendo a proporção original das classes com `stratify=y` :

In [8]:
```
X_treino, X_teste, y_treino, y_teste = train_test_split(
```

```
    X,
    y,
    test_size=0.2,
    random_state=RANDOM_STATE,
    stratify=y
)
```

Padronizando as variáveis numéricas

Diferente das árvores, que são **insensíveis à escala** das variáveis, redes neurais são fortemente afetadas por variáveis em ordens de grandeza muito diferentes (por exemplo, idade em dezenas e valor de transação em milhares). Sem padronização, o gradiente descendente tem dificuldade em convergir e o treinamento se torna instável.

Aplicando a padronização. O `StandardScaler` aprende a escala usando apenas o treino e depois aplica a mesma transformação no teste:

In [9]:
```
from sklearn.preprocessing import StandardScaler
```

In [10]:
```
scaler = StandardScaler()

X_treino_scaled = scaler.fit_transform(X_treino)
X_teste_scaled = scaler.transform(X_teste)
```

Inspecionando rapidamente o resultado da padronização — os valores agora têm média próxima de zero e desvio padrão próximo de um:

In [11]:
```
X_treino_scaled
```

Out[11]:
```
array([[ 1.93342791, -0.78208084, -0.26608479, ..., -0.10419214,
        -0.04160737, -0.23821013],
       [-0.75259779,  1.33452938, -0.26608479, ..., -0.10419214,
        -0.04160737, -0.23821013],
       [-0.59655062, -0.78208084,  1.2824446 , ..., -0.10419214,
        -0.04160737, -0.23821013],
       ...,
       [-0.63480958, -0.40856139, -0.26608479, ..., -0.10419214,
        -0.04160737, -0.23821013],
       [-0.65938479,  1.58354234, -0.26608479, ..., -0.10419214,
        -0.04160737, -0.23821013],
       [-0.80274183,  2.20607476, -1.81461419, ..., -0.10419214,
        -0.04160737, -0.23821013]], shape=(8101, 38))
```

In [12]:
```
X_teste_scaled
```

Out[12]:
```
array([[-0.81636881,  0.21397102,  1.2824446 , ..., -0.10419214,
        -0.04160737, -0.23821013],
       [ 1.07772254, -0.15954843,  1.2824446 , ..., -0.10419214,
        -0.04160737, -0.23821013],
       [-0.75448772,  0.21397102, -0.26608479, ..., -0.10419214,
        -0.04160737, -0.23821013],
       ...,
       [-0.58121621, -1.1556003 ,  2.0567093 , ..., -0.10419214,
        -0.04160737,  4.19797433],
       [ 0.90705476, -1.03109381, -0.26608479, ..., -0.10419214,
        -0.04160737, -0.23821013],
       [-0.80173806, -0.15954843,  0.5081799 , ..., -0.10419214,
        -0.04160737, -0.23821013]], shape=(2026, 38))
```

Lidando com o desbalanceamento das classes

Como existem menos clientes `Inativo`, vamos calcular pesos simples para as classes. Isso ajuda a rede a não ignorar a classe minoritária durante o treinamento:

In [13]:
```
from sklearn.utils.class_weight import compute_class_weight
```

In [14]:
```
pesos = compute_class_weight(
    class_weight='balanced',
```

```
    classes=np.array([0, 1]),
    y=y_treino
)

class_weight = {0: pesos[0], 1: pesos[1]}
class_weight
```

Out[14]:
```
{0: np.float64(0.5957493749080747), 1: np.float64(3.1109831029185866)}
```

Montando uma rede neural simples

A rede abaixo tem apenas uma camada intermediária pequena e uma saída para prever a probabilidade de churn. A saída será interpretada como:

- perto de 0: maior chance de cliente `Ativo` ;
- perto de 1: maior chance de cliente `Inativo` .

In [15]:
```
modelo_rede = keras.Sequential([
    keras.layers.Input(shape=(X_treino_scaled.shape[1],)),
    keras.layers.Dense(16, activation='relu'),
    keras.layers.Dense(1, activation='sigmoid')
])

modelo_rede.compile(
    optimizer='adam',
    loss='binary_crossentropy',
    metrics=[
        'recall',
        keras.metrics.AUC(name='auc')
    ]
)

modelo_rede.summary()
```

```
WARNING:tensorflow:TensorFlow GPU support is not available on native Windows for TensorFlow >= 2.11. Even if CUDA/cuDNN are ins
talled, GPU will not be used. Please use WSL2 or the TensorFlow-DirectML plugin.
Model: "sequential"
```

Layer (type)	Output Shape	Param #
dense (Dense)	(None, 16)	624
dense_1 (Dense)	(None, 1)	17

Total params: 641 (2.50 KB)
Trainable params: 641 (2.50 KB)
Non-trainable params: 0 (0.00 B)

Treinando a rede

In [16]:
```
early_stopping = keras.callbacks.EarlyStopping(
    monitor='val_auc',
    mode='max',
    patience=5,
    restore_best_weights=True
)

historico = modelo_rede.fit(
    X_treino_scaled,
    y_treino,
    validation_split=0.2,
    epochs=50,
    batch_size=32,
    callbacks=[early_stopping],
    class_weight=class_weight,
    verbose=1
)
```

```
Epoch 1/50
  1/203 ━━━━━━━━━━━━━━━━━━━━ 2:57 879ms/step - auc: 0.6339 - loss: 0.6158 - recall: 0.5000

 42/203 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - auc: 0.4674 - loss: 0.7990 - recall: 0.4036

 86/203 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - auc: 0.5275 - loss: 0.7505 - recall: 0.4586

131/203 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - auc: 0.5933 - loss: 0.7181 - recall: 0.5377
```

```
172/203 ━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - auc: 0.6388 - loss: 0.6765 - recall: 0.5876

203/203 ━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - auc: 0.6676 - loss: 0.6566 - recall: 0.6121 - val_auc: 0.8243 - val_loss: 0.5589 - v
al_recall: 0.7752
Epoch 2/50
  1/203 ━━━━━━━━━━━━━━━━━━━ 9s 46ms/step - auc: 0.7991 - loss: 0.4669 - recall: 0.7500

 44/203 ━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - auc: 0.8665 - loss: 0.4936 - recall: 0.8304

 87/203 ━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - auc: 0.8678 - loss: 0.4868 - recall: 0.8322

133/203 ━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - auc: 0.8652 - loss: 0.4900 - recall: 0.8223

176/203 ━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - auc: 0.8699 - loss: 0.4739 - recall: 0.8258

203/203 ━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - auc: 0.8715 - loss: 0.4702 - recall: 0.8276 - val_auc: 0.8832 - val_loss: 0.4661 - v
al_recall: 0.8256
Epoch 3/50
  1/203 ━━━━━━━━━━━━━━━━━━━ 4s 22ms/step - auc: 0.8214 - loss: 0.4395 - recall: 0.7500

 44/203 ━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - auc: 0.9022 - loss: 0.4118 - recall: 0.8304

 86/203 ━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - auc: 0.9028 - loss: 0.4097 - recall: 0.8412

128/203 ━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - auc: 0.8930 - loss: 0.4254 - recall: 0.8195

171/203 ━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - auc: 0.8954 - loss: 0.4175 - recall: 0.8264

203/203 ━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - auc: 0.8971 - loss: 0.4135 - recall: 0.8314 - val_auc: 0.9000 - val_loss: 0.4211 - v
al_recall: 0.8295
Epoch 4/50
  1/203 ━━━━━━━━━━━━━━━━━━━ 3s 18ms/step - auc: 0.8214 - loss: 0.4336 - recall: 0.7500

 43/203 ━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - auc: 0.9147 - loss: 0.3804 - recall: 0.8407

 86/203 ━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - auc: 0.9144 - loss: 0.3792 - recall: 0.8501

130/203 ━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - auc: 0.9065 - loss: 0.3962 - recall: 0.8326

173/203 ━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - auc: 0.9076 - loss: 0.3894 - recall: 0.8372
```

```
203/203 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - auc: 0.9092 - loss: 0.3861 - recall: 0.8381 - val_auc: 0.9104 - val_loss: 0.3947 - v
al_recall: 0.8411
Epoch 5/50
  1/203 ━━━━━━━━━━━━━━━━━━━━ 1:01 305ms/step - auc: 0.8214 - loss: 0.4304 - recall: 0.7500

 43/203 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - auc: 0.9228 - loss: 0.3606 - recall: 0.8451

 88/203 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - auc: 0.9198 - loss: 0.3657 - recall: 0.8485

133/203 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - auc: 0.9159 - loss: 0.3758 - recall: 0.8392

177/203 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - auc: 0.9163 - loss: 0.3696 - recall: 0.8441

203/203 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - auc: 0.9178 - loss: 0.3672 - recall: 0.8458 - val_auc: 0.9182 - val_loss: 0.3758 - v
al_recall: 0.8488
Epoch 6/50
  1/203 ━━━━━━━━━━━━━━━━━━━━ 4s 20ms/step - auc: 0.8348 - loss: 0.4292 - recall: 0.7500

 42/203 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - auc: 0.9293 - loss: 0.3475 - recall: 0.8296

 85/203 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - auc: 0.9292 - loss: 0.3434 - recall: 0.8477

128/203 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - auc: 0.9217 - loss: 0.3612 - recall: 0.8388

165/203 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - auc: 0.9231 - loss: 0.3568 - recall: 0.8455

203/203 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - auc: 0.9248 - loss: 0.3519 - recall: 0.8487 - val_auc: 0.9242 - val_loss: 0.3611 - v
al_recall: 0.8450
Epoch 7/50
  1/203 ━━━━━━━━━━━━━━━━━━━━ 4s 21ms/step - auc: 0.8482 - loss: 0.4265 - recall: 0.7500

 41/203 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - auc: 0.9325 - loss: 0.3389 - recall: 0.8349

 82/203 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - auc: 0.9347 - loss: 0.3290 - recall: 0.8500

123/203 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - auc: 0.9288 - loss: 0.3455 - recall: 0.8490

163/203 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - auc: 0.9303 - loss: 0.3410 - recall: 0.8525
```

```
203/203 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - auc: 0.9306 - loss: 0.3387 - recall: 0.8544 - val_auc: 0.9292 - val_loss: 0.3491 - v
al_recall: 0.8605
Epoch 8/50
  1/203 ━━━━━━━━━━━━━━━━━━━━ 4s 21ms/step - auc: 0.8571 - loss: 0.4224 - recall: 0.7500

 40/203 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - auc: 0.9371 - loss: 0.3308 - recall: 0.8525

 81/203 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - auc: 0.9397 - loss: 0.3176 - recall: 0.8630

122/203 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - auc: 0.9352 - loss: 0.3306 - recall: 0.8631

163/203 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - auc: 0.9354 - loss: 0.3290 - recall: 0.8630

203/203 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - auc: 0.9356 - loss: 0.3269 - recall: 0.8611 - val_auc: 0.9326 - val_loss: 0.3385 - v
al_recall: 0.8682
Epoch 9/50
  1/203 ━━━━━━━━━━━━━━━━━━━━ 59s 294ms/step - auc: 0.8616 - loss: 0.4194 - recall: 0.7500

 42/203 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - auc: 0.9423 - loss: 0.3146 - recall: 0.8520

 84/203 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - auc: 0.9431 - loss: 0.3102 - recall: 0.8673

123/203 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - auc: 0.9381 - loss: 0.3231 - recall: 0.8644

163/203 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - auc: 0.9395 - loss: 0.3187 - recall: 0.8653

203/203 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - auc: 0.9396 - loss: 0.3168 - recall: 0.8678 - val_auc: 0.9357 - val_loss: 0.3291 - v
al_recall: 0.8643
Epoch 10/50
  1/203 ━━━━━━━━━━━━━━━━━━━━ 4s 23ms/step - auc: 0.8795 - loss: 0.4187 - recall: 0.7500

 43/203 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - auc: 0.9455 - loss: 0.3046 - recall: 0.8540

 84/203 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - auc: 0.9463 - loss: 0.3015 - recall: 0.8673

126/203 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - auc: 0.9410 - loss: 0.3163 - recall: 0.8627

168/203 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - auc: 0.9416 - loss: 0.3124 - recall: 0.8664
```

```
203/203 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - auc: 0.9431 - loss: 0.3076 - recall: 0.8726 - val_auc: 0.9381 - val_loss: 0.3209 - v
al_recall: 0.8721
Epoch 11/50
  1/203 ━━━━━━━━━━━━━━━━━━━━ 4s 20ms/step - auc: 0.8795 - loss: 0.4207 - recall: 0.7500

 43/203 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - auc: 0.9482 - loss: 0.2974 - recall: 0.8628

 85/203 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - auc: 0.9487 - loss: 0.2942 - recall: 0.8750

126/203 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - auc: 0.9443 - loss: 0.3077 - recall: 0.8642

168/203 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - auc: 0.9447 - loss: 0.3040 - recall: 0.8687

203/203 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - auc: 0.9463 - loss: 0.2992 - recall: 0.8755 - val_auc: 0.9409 - val_loss: 0.3120 - v
al_recall: 0.8760
Epoch 12/50
  1/203 ━━━━━━━━━━━━━━━━━━━━ 1:00 299ms/step - auc: 0.8839 - loss: 0.4177 - recall: 0.7500

 43/203 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - auc: 0.9509 - loss: 0.2901 - recall: 0.8717

 84/203 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - auc: 0.9518 - loss: 0.2860 - recall: 0.8787

126/203 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - auc: 0.9473 - loss: 0.2994 - recall: 0.8701

168/203 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - auc: 0.9477 - loss: 0.2959 - recall: 0.8744

203/203 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - auc: 0.9493 - loss: 0.2909 - recall: 0.8803 - val_auc: 0.9436 - val_loss: 0.3035 - v
al_recall: 0.8721
Epoch 13/50
  1/203 ━━━━━━━━━━━━━━━━━━━━ 3s 19ms/step - auc: 0.8884 - loss: 0.4200 - recall: 0.7500

 42/203 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - auc: 0.9532 - loss: 0.2843 - recall: 0.8789

 84/203 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - auc: 0.9545 - loss: 0.2783 - recall: 0.8833

125/203 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - auc: 0.9502 - loss: 0.2907 - recall: 0.8761

165/203 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - auc: 0.9512 - loss: 0.2867 - recall: 0.8815
```

```
203/203 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - auc: 0.9524 - loss: 0.2823 - recall: 0.8860 - val_auc: 0.9459 - val_loss: 0.2946 - v
al_recall: 0.8721
Epoch 14/50
  1/203 ━━━━━━━━━━━━━━━━━━━━ 4s 20ms/step - auc: 0.8929 - loss: 0.4177 - recall: 0.7500

 42/203 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - auc: 0.9558 - loss: 0.2766 - recall: 0.8834

 83/203 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - auc: 0.9562 - loss: 0.2718 - recall: 0.8832

122/203 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - auc: 0.9545 - loss: 0.2784 - recall: 0.8834

163/203 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - auc: 0.9549 - loss: 0.2765 - recall: 0.8876

203/203 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - auc: 0.9552 - loss: 0.2740 - recall: 0.8908 - val_auc: 0.9482 - val_loss: 0.2868 - v
al_recall: 0.8721
Epoch 15/50
  1/203 ━━━━━━━━━━━━━━━━━━━━ 4s 22ms/step - auc: 0.8929 - loss: 0.4168 - recall: 0.7500

 43/203 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - auc: 0.9586 - loss: 0.2671 - recall: 0.8938

 85/203 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - auc: 0.9593 - loss: 0.2631 - recall: 0.8955

127/203 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - auc: 0.9559 - loss: 0.2744 - recall: 0.8854

168/203 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - auc: 0.9562 - loss: 0.2719 - recall: 0.8938

203/203 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - auc: 0.9580 - loss: 0.2661 - recall: 0.8966 - val_auc: 0.9501 - val_loss: 0.2802 - v
al_recall: 0.8721
Epoch 16/50
  1/203 ━━━━━━━━━━━━━━━━━━━━ 4s 20ms/step - auc: 0.9018 - loss: 0.4126 - recall: 0.7500

 44/203 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - auc: 0.9613 - loss: 0.2584 - recall: 0.8957

 85/203 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - auc: 0.9615 - loss: 0.2564 - recall: 0.8932

127/203 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - auc: 0.9584 - loss: 0.2670 - recall: 0.8854

168/203 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - auc: 0.9586 - loss: 0.2647 - recall: 0.8938
```

```
203/203 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - auc: 0.9604 - loss: 0.2588 - recall: 0.8985 - val_auc: 0.9516 - val_loss: 0.2749 - v
al_recall: 0.8760
Epoch 17/50
  1/203 ━━━━━━━━━━━━━━━━━━━━ 4s 22ms/step - auc: 0.9107 - loss: 0.4067 - recall: 0.7500

 41/203 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - auc: 0.9616 - loss: 0.2587 - recall: 0.8899

 82/203 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - auc: 0.9636 - loss: 0.2485 - recall: 0.8905

125/203 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - auc: 0.9604 - loss: 0.2604 - recall: 0.8882

166/203 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - auc: 0.9612 - loss: 0.2564 - recall: 0.8949

203/203 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - auc: 0.9624 - loss: 0.2521 - recall: 0.8975 - val_auc: 0.9529 - val_loss: 0.2698 - v
al_recall: 0.8721
Epoch 18/50
  1/203 ━━━━━━━━━━━━━━━━━━━━ 4s 21ms/step - auc: 0.9196 - loss: 0.4026 - recall: 0.7500

 42/203 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - auc: 0.9643 - loss: 0.2491 - recall: 0.9103

 84/203 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - auc: 0.9654 - loss: 0.2437 - recall: 0.9062

126/203 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - auc: 0.9624 - loss: 0.2540 - recall: 0.8985

167/203 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - auc: 0.9629 - loss: 0.2503 - recall: 0.9023

203/203 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - auc: 0.9642 - loss: 0.2458 - recall: 0.9052 - val_auc: 0.9536 - val_loss: 0.2660 - v
al_recall: 0.8760
Epoch 19/50
  1/203 ━━━━━━━━━━━━━━━━━━━━ 4s 20ms/step - auc: 0.9196 - loss: 0.3995 - recall: 0.7500

 42/203 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - auc: 0.9662 - loss: 0.2429 - recall: 0.9058

 84/203 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - auc: 0.9670 - loss: 0.2378 - recall: 0.9039

126/203 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - auc: 0.9642 - loss: 0.2481 - recall: 0.8985

167/203 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - auc: 0.9646 - loss: 0.2445 - recall: 0.9034
```

```
203/203 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - auc: 0.9659 - loss: 0.2400 - recall: 0.9071 - val_auc: 0.9543 - val_loss: 0.2633 - v
al_recall: 0.8837
Epoch 20/50
  1/203 ━━━━━━━━━━━━━━━━━━━━ 4s 21ms/step - auc: 0.9196 - loss: 0.3935 - recall: 0.7500

 42/203 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - auc: 0.9678 - loss: 0.2371 - recall: 0.9283

 83/203 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - auc: 0.9677 - loss: 0.2339 - recall: 0.9159

125/203 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - auc: 0.9656 - loss: 0.2424 - recall: 0.9063

166/203 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - auc: 0.9664 - loss: 0.2385 - recall: 0.9122

203/203 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - auc: 0.9673 - loss: 0.2347 - recall: 0.9148 - val_auc: 0.9548 - val_loss: 0.2611 - v
al_recall: 0.8837
Epoch 21/50
  1/203 ━━━━━━━━━━━━━━━━━━━━ 4s 20ms/step - auc: 0.9196 - loss: 0.3894 - recall: 0.7500

 43/203 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - auc: 0.9697 - loss: 0.2295 - recall: 0.9381

 84/203 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - auc: 0.9697 - loss: 0.2275 - recall: 0.9199

124/203 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - auc: 0.9670 - loss: 0.2373 - recall: 0.9085

165/203 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - auc: 0.9677 - loss: 0.2340 - recall: 0.9141

203/203 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - auc: 0.9686 - loss: 0.2301 - recall: 0.9186 - val_auc: 0.9552 - val_loss: 0.2592 - v
al_recall: 0.8760
Epoch 22/50
  1/203 ━━━━━━━━━━━━━━━━━━━━ 58s 289ms/step - auc: 0.9196 - loss: 0.3852 - recall: 0.7500

 42/203 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - auc: 0.9703 - loss: 0.2273 - recall: 0.9417

 84/203 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - auc: 0.9707 - loss: 0.2234 - recall: 0.9314

127/203 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - auc: 0.9682 - loss: 0.2327 - recall: 0.9211

168/203 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - auc: 0.9681 - loss: 0.2315 - recall: 0.9224
```

```
203/203 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - auc: 0.9697 - loss: 0.2258 - recall: 0.9262 - val_auc: 0.9556 - val_loss: 0.2571 - v
al_recall: 0.8721
Epoch 23/50
  1/203 ━━━━━━━━━━━━━━━━━━━━ 4s 23ms/step - auc: 0.9196 - loss: 0.3808 - recall: 0.7500

 42/203 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - auc: 0.9714 - loss: 0.2228 - recall: 0.9417

 84/203 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - auc: 0.9716 - loss: 0.2194 - recall: 0.9382

126/203 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - auc: 0.9693 - loss: 0.2290 - recall: 0.9254

167/203 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - auc: 0.9696 - loss: 0.2261 - recall: 0.9264

203/203 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - auc: 0.9706 - loss: 0.2220 - recall: 0.9291 - val_auc: 0.9559 - val_loss: 0.2554 - v
al_recall: 0.8721
Epoch 24/50
  1/203 ━━━━━━━━━━━━━━━━━━━━ 1:02 312ms/step - auc: 0.9196 - loss: 0.3793 - recall: 0.7500

 43/203 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - auc: 0.9728 - loss: 0.2162 - recall: 0.9425

 84/203 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - auc: 0.9725 - loss: 0.2159 - recall: 0.9405

125/203 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - auc: 0.9700 - loss: 0.2254 - recall: 0.9245

163/203 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - auc: 0.9713 - loss: 0.2202 - recall: 0.9274

203/203 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - auc: 0.9714 - loss: 0.2185 - recall: 0.9282

203/203 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - auc: 0.9714 - loss: 0.2185 - recall: 0.9282 - val_auc: 0.9561 - val_loss: 0.2542 - v
al_recall: 0.8721
Epoch 25/50
  1/203 ━━━━━━━━━━━━━━━━━━━━ 1:01 305ms/step - auc: 0.9196 - loss: 0.3768 - recall: 0.7500

 40/203 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - auc: 0.9723 - loss: 0.2204 - recall: 0.9447

 82/203 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - auc: 0.9729 - loss: 0.2121 - recall: 0.9452

124/203 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - auc: 0.9709 - loss: 0.2220 - recall: 0.9284

166/203 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - auc: 0.9714 - loss: 0.2187 - recall: 0.9307
```

```
203/203 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - auc: 0.9722 - loss: 0.2154 - recall: 0.9310 - val_auc: 0.9562 - val_loss: 0.2531 - v
al_recall: 0.8721
Epoch 26/50
  1/203 ━━━━━━━━━━━━━━━━━━━━ 1:02 311ms/step - auc: 0.9196 - loss: 0.3750 - recall: 0.7500

 42/203 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - auc: 0.9738 - loss: 0.2121 - recall: 0.9462

 84/203 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - auc: 0.9738 - loss: 0.2097 - recall: 0.9428

126/203 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - auc: 0.9717 - loss: 0.2188 - recall: 0.9343

167/203 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - auc: 0.9719 - loss: 0.2161 - recall: 0.9345

203/203 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - auc: 0.9728 - loss: 0.2124 - recall: 0.9349 - val_auc: 0.9563 - val_loss: 0.2522 - v
al_recall: 0.8721
Epoch 27/50
  1/203 ━━━━━━━━━━━━━━━━━━━━ 4s 22ms/step - auc: 0.9196 - loss: 0.3728 - recall: 0.7500

 42/203 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - auc: 0.9743 - loss: 0.2095 - recall: 0.9417

 84/203 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - auc: 0.9744 - loss: 0.2071 - recall: 0.9382

125/203 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - auc: 0.9723 - loss: 0.2161 - recall: 0.9320

169/203 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - auc: 0.9723 - loss: 0.2143 - recall: 0.9339

203/203 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - auc: 0.9735 - loss: 0.2097 - recall: 0.9349 - val_auc: 0.9562 - val_loss: 0.2511 - v
al_recall: 0.8682
Epoch 28/50
  1/203 ━━━━━━━━━━━━━━━━━━━━ 3s 19ms/step - auc: 0.9196 - loss: 0.3694 - recall: 0.7500

 42/203 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - auc: 0.9749 - loss: 0.2066 - recall: 0.9417

 81/203 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - auc: 0.9749 - loss: 0.2033 - recall: 0.9423

122/203 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - auc: 0.9735 - loss: 0.2108 - recall: 0.9347

164/203 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - auc: 0.9738 - loss: 0.2089 - recall: 0.9347
```

```
203/203 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - auc: 0.9740 - loss: 0.2072 - recall: 0.9339 - val_auc: 0.9562 - val_loss: 0.2508 - v
al_recall: 0.8682
Epoch 29/50
  1/203 ━━━━━━━━━━━━━━━━━━━━ 4s 20ms/step - auc: 0.9196 - loss: 0.3677 - recall: 0.7500

 45/203 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - auc: 0.9757 - loss: 0.2006 - recall: 0.9487

 89/203 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - auc: 0.9743 - loss: 0.2069 - recall: 0.9404

137/203 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - auc: 0.9739 - loss: 0.2094 - recall: 0.9366

182/203 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - auc: 0.9743 - loss: 0.2057 - recall: 0.9372

203/203 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - auc: 0.9745 - loss: 0.2049 - recall: 0.9368 - val_auc: 0.9561 - val_loss: 0.2505 - v
al_recall: 0.8721
Epoch 30/50
  1/203 ━━━━━━━━━━━━━━━━━━━━ 3s 19ms/step - auc: 0.9196 - loss: 0.3667 - recall: 0.7500

 47/203 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - auc: 0.9758 - loss: 0.1992 - recall: 0.9595

 92/203 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - auc: 0.9748 - loss: 0.2042 - recall: 0.9441

137/203 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - auc: 0.9743 - loss: 0.2074 - recall: 0.9394

182/203 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - auc: 0.9747 - loss: 0.2035 - recall: 0.9394

203/203 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - auc: 0.9749 - loss: 0.2028 - recall: 0.9387 - val_auc: 0.9560 - val_loss: 0.2500 - v
al_recall: 0.8760
Epoch 31/50
  1/203 ━━━━━━━━━━━━━━━━━━━━ 3s 19ms/step - auc: 0.9286 - loss: 0.3645 - recall: 0.7500

 41/203 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - auc: 0.9758 - loss: 0.2018 - recall: 0.9541

 84/203 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - auc: 0.9763 - loss: 0.1980 - recall: 0.9428

129/203 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - auc: 0.9746 - loss: 0.2060 - recall: 0.9371

172/203 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - auc: 0.9742 - loss: 0.2052 - recall: 0.9382
```

```
203/203 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - auc: 0.9754 - loss: 0.2009 - recall: 0.9397 - val_auc: 0.9559 - val_loss: 0.2500 - v
al_recall: 0.8760
```

Visualizando a evolução da rede ao longo do treinamento. A ideia é observar se o desempenho no treino e na validação caminham de forma parecida:

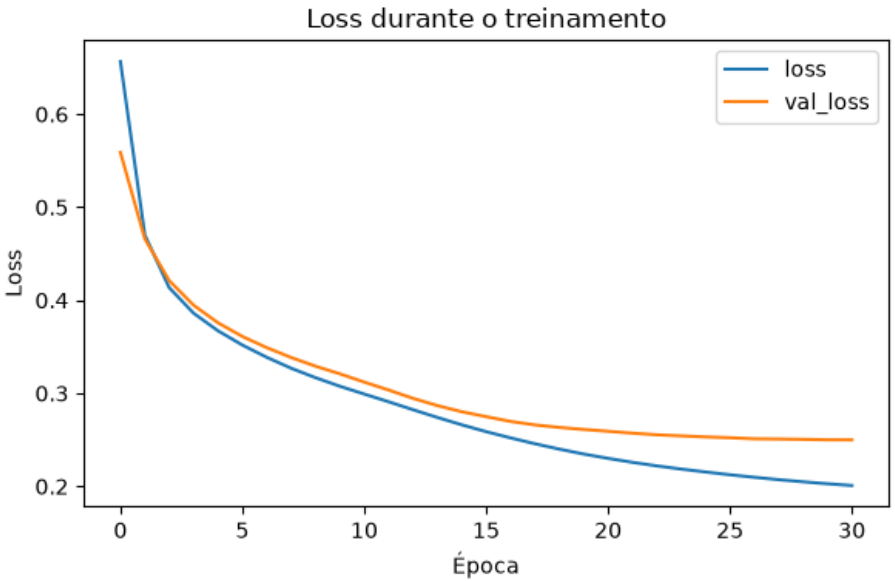
In [17]:
```
historico_df = pd.DataFrame(historico.history)

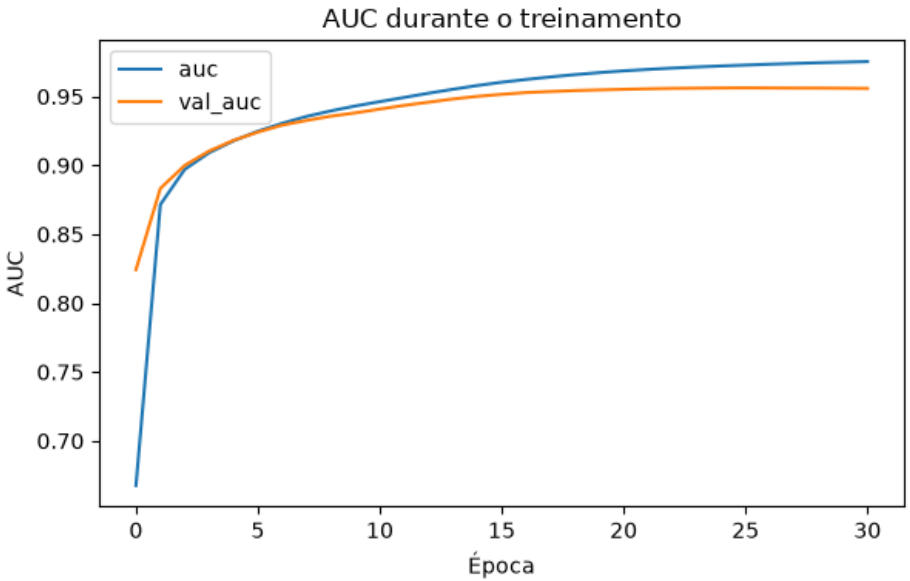
fig, axes = plt.subplots(1, 2, figsize=(12, 4))

historico_df[['loss', 'val_loss']].plot(ax=axes[0])
axes[0].set_title('Loss durante o treinamento')
axes[0].set_xlabel('Época')
axes[0].set_ylabel('Loss')

historico_df[['auc', 'val_auc']].plot(ax=axes[1])
axes[1].set_title('AUC durante o treinamento')
axes[1].set_xlabel('Época')
axes[1].set_ylabel('AUC')

plt.tight_layout()
plt.show()
```





Avaliando no conjunto de teste

Agora vamos avaliar a rede em dados que ela não viu durante o treinamento. Como a rede retorna probabilidades, usamos o ponto de corte de 0,5 para transformar a probabilidade em classe prevista:

In [18]:
```
probabilidades_teste = modelo_rede.predict(X_teste_scaled).ravel()
predicoes_teste = (probabilidades_teste >= 0.5).astype(int)

acuracia_teste = accuracy_score(y_teste, predicoes_teste)
roc_auc_teste = roc_auc_score(y_teste, probabilidades_teste)

print(f'Acurácia em teste: {acuracia_teste:.3f}')
print(f'ROC-AUC em teste:  {roc_auc_teste:.3f}')
print()
print('Relatório de classificação:')
print(classification_report(
    y_teste,
    predicoes_teste,
    target_names=target_names,
    digits=3,
```

```
    zero_division=0
))
```

```
 1/64 ━━━━━━━━━━━━━━━━━━━━ 2s 41ms/step

64/64 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step

64/64 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step
Acurácia em teste: 0.897
ROC-AUC em teste:  0.954

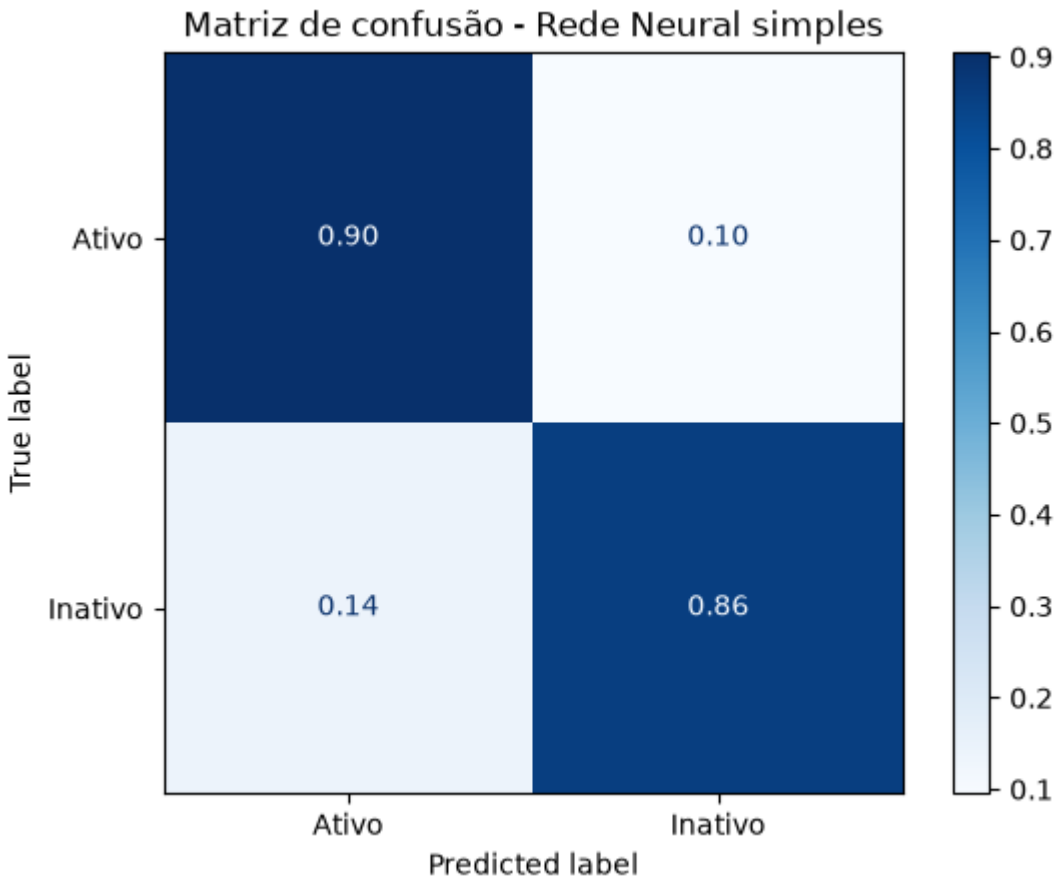
Relatório de classificação:
              precision    recall  f1-score   support

       Ativo      0.971     0.904     0.936      1701
     Inativo      0.631     0.858     0.728       325

    accuracy                          0.897      2026
   macro avg      0.801     0.881     0.832      2026
weighted avg      0.916     0.897     0.903      2026
```

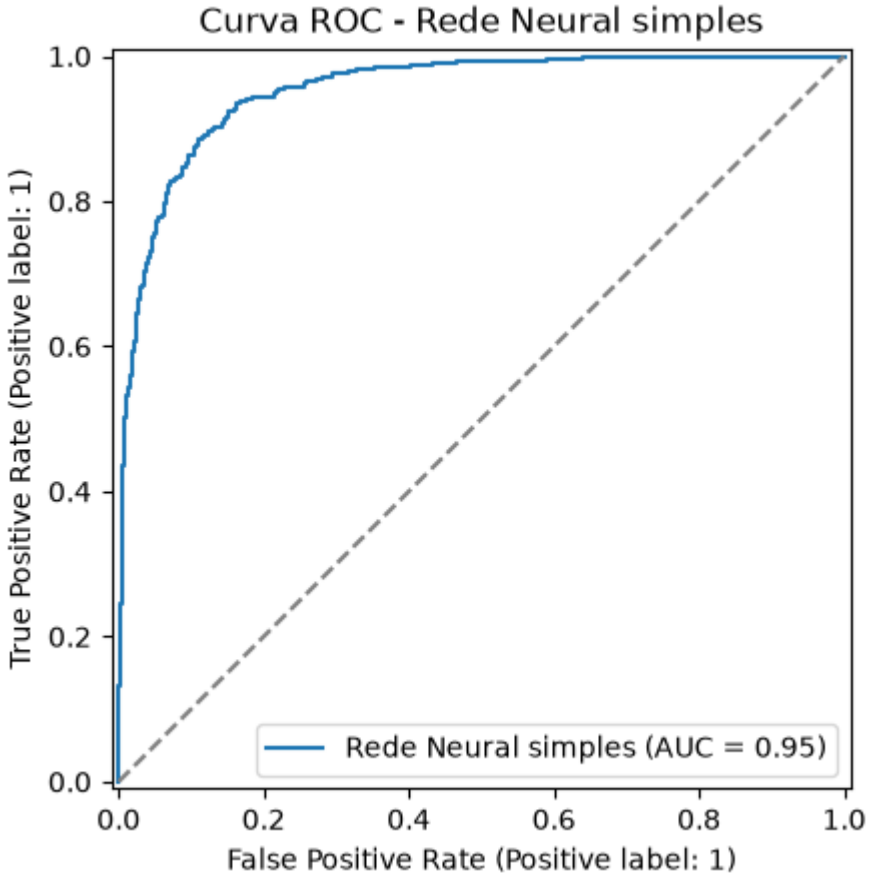
Matriz de confusão normalizada para observar os acertos e erros em cada classe:

In [19]:
```
ConfusionMatrixDisplay.from_predictions(
    y_teste,
    predicoes_teste,
    display_labels=target_names,
    normalize='true',
    cmap='Blues',
    values_format='.2f'
)
plt.title('Matriz de confusão - Rede Neural simples')
plt.show()
```



Curva ROC para avaliar a capacidade da rede de separar clientes ativos e inativos:

In [20]:
```
RocCurveDisplay.from_predictions(
    y_teste,
    probabilidades_teste,
    name='Rede Neural simples'
)
plt.plot([0, 1], [0, 1], linestyle='--', color='gray')
plt.title('Curva ROC - Rede Neural simples')
plt.show()
```


Curva ROC - Rede Neural simples
True Positive Rate (Positive label: 1)
False Positive Rate (Positive label: 1)
Rede Neural simples (AUC = 0.95)
0.0
0.2
0.4
0.6
0.8
1.0